kai-leverage verification report

📝 Summary
- Key Findings
- Verification Results
- Manual Audit Findings
- Remediation Summary

Scope and Limitations
- Scope
- Limitations
- Legal Disclaimer

⚠️ Issues
- 🔴 C-1: A liquidity provider can steal from pool via flash loan
  - 🔧 Recommendation
  - ✅ Remediation
- 🔴 C-2: max_liquidity_factor computation is incorrect
  - 🔧 Recommendation
  - ✅ Remediation
- 🔴 C-3: Missing assert in liquidate_col_x /liquidate_col_y
  - 🔧 Recommendation
  - ✅ Remediation
- 🟡 L-1: Missing increment of i
  - 🔧 Recommendation
  - ✅ Remediation
- 🟡 L-2: Bug in condition when checking we borrowed enough
  - 🔧 Recommendation
  - ✅ Remediation
- 🟡 L-3: Incorrect treasury empty check
  - 🔧 Recommendation
  - ✅ Remediation
- 🧹 R-1: Split debt_bag into its two components
  - 🔧 Recommendation

□ Remediation
⊢ Verification summary
supply_pool.move
Rounding
Bookkeeping property
position_model.move
Avoiding sensitive aborts
Other properties

📝 Summary

Asymptotic has conducted a comprehensive security audit and verification of the `kai-leverage` project, examining commit d6cdbbe of the codebase. Due to limitations in the Move Prover for post-Move 2024 code, we performed the verification of the code right before it was adapted for Move 2024. Finally, we make the verification artifacts available for CI integration down the line.

🔗 Github repository: sui-smart-contracts
Inspected commit: d6cdbbedc408b9a18abfdc038b13211cf6cb586c (not available in open-sourced repo)
We audited an early pre kai-leverage-v1 version
Verification commit: Commit b3de085
Verification artifacts: Commit c4fd001 (private repo)

Key Findings

- **Critical Vulnerabilities**: We identified three critical vulnerabilities (C-1, C-2, and C-3), with C-1 being a true vulnerability and C-2 and C-3 intentionally planted to verify audit quality.
  - C-1: A flash loan exploit allows liquidity providers to drain the pool. Remediated, and flash loan functionality was also subsequently removed from the contract before publishing.
  - C-2: Incorrect computation of `max_liquidity_factor`. Remediated.
  - C-3: Missing assertion in liquidation functions. Remediated.
- **Medium and Low Severity Issues**: Three low (L-1, L-2, L-3) severity issues were discovered, primarily related to incorrect checks and minor logic errors.
- **Refactoring Suggestions**: We proposed one refactoring suggestion (R-1) to improve code structure and type safety.

Verification Results

Formal verification was performed on key components of `supply_pool.move` and `position_model.move`. This process helped identify several vulnerabilities and confirmed critical properties of the system, including:

- Correct rounding behavior in `EquityTreasury` and `DebtTreasury`.
- Bookkeeping property in `SupplyPool`.
- Non-aborting behavior of critical functions in `position_model.move`.

Manual Audit Findings

The manual audit focused on access control, error handling, and code quality. While many aspects were implemented correctly, we identified areas requiring further clarification or improvement, particularly in access management and error handling.

Remediation Summary

The kai-leverage project team has addressed most of the identified issues. The critical flash loan vulnerability (C-1) was fixed by correctly calculating the interest added to the pool's value; furthermore, the flash loan functionality was fully removed from the smart contract before it was published. The intentionally planted vulnerabilities (C-2 and C-3) were also remediated. Of the low-severity issues, L-1 and L-2 were resolved by correcting the loop increment and borrowing condition respectively.L-3, regarding the incorrect treasury empty check, was not addressed, but the code is still safe due to other layers. The refactoring suggestion (R-1) to split the `debt_bag` into separate components was not implemented.

Scope and Limitations

Scope

This audit covered the following components of the `kai-leverage` project:

1. Smart Contracts:
    - `supply_pool.move`
    - `position_model.move`
    - `access_init.move`
    - `access.move`
    - `position_core.move`
2. Commit Hash: d6cdbbe

3. Specific Focus Areas:
    - Access control mechanisms
    - Early abort and error codes
    - Liquidity management
    - Flash loan functionality
    - Liquidation processes
    - Interest calculation and fee management
4. Verification:
    - Formal verification of specific properties in `supply_pool.move` and `position_model.move`

Limitations

1. Formal Verification:
    - While some properties were formally verified, not all functions and interactions were covered by formal verification.
    - **In particular, `cetus.move` was not verified because it would require modeling of Uniswap v3 and oracle code.**
2. Time Constraints:
    - The audit was conducted over a three-week period and may not have uncovered all potential vulnerabilities.
3. Excluded Components:
    - External dependencies and libraries were not audited unless explicitly mentioned.
    - The Uniswap v3 and oracle implementations were not fully modeled or audited.
4. Dynamic Analysis:
    - No dynamic analysis or fuzzing was performed as part of this audit.
5. Cryptographic Implementations:
    - Detailed analysis of cryptographic implementations was out of scope.

6. Economic Attacks:
    - The audit focused on technical vulnerabilities and did not extensively model potential economic attack vectors.
7. Deployment and Operational Security:
    - The audit did not cover operational security practices or deployment processes.
8. Regulatory Compliance:
    - Assessment of compliance with financial regulations or legal requirements was not part of this audit.
9. Future Changes:
    - This audit report reflects the state of the project at the specified commit hash. Future changes or updates may introduce new vulnerabilities.

We recommend continuous security assessments and thorough testing of any changes or additions to the codebase.

Legal Disclaimer

This security audit report (the "Report") is provided on an as-is basis and does not constitute a guarantee or warranty of any kind. The information contained in this Report is for informational purposes only and should not be considered exhaustive or as legal, financial, or professional advice.

1. No Warranty:
   Asymptotic (the "Auditor") makes no representations or warranties regarding the accuracy, completeness, or fitness for a particular purpose of this Report. The Auditor expressly disclaims any and all warranties, whether express, implied, or statutory.

2. Limitation of Liability:
   In no event shall the Auditor be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services, loss of use, data, or profits, or business interruption) arising in any way out of the use of this Report, even if advised of the possibility of such damage.
3. Use at Own Risk:
   Any use or reliance on this Report or the information contained herein is at the user's own risk. The Report does not constitute investment advice, and users should exercise their own judgment and perform their own due diligence before making any investment or business decisions based on this Report.
4. No Endorsement:
   This Report does not constitute an endorsement of the audited project or its associated tokens, coins, or any other digital assets.
5. Future Vulnerabilities:
   The audit and this Report are limited to the scope and time frame specified. Future changes to the codebase, deployment environment, or external dependencies may introduce new vulnerabilities not covered by this Report.
6. Third-Party Reliance:
   This Report is intended solely for the use of the client (kai-leverage project) and Asymptotic. Any third-party use or reliance on this Report is at their own risk.
7. Confidentiality:
   The contents of this Report are confidential and should not be disclosed without the express written permission of both the client and Asymptotic.
8. Governing Law:
   This disclaimer and any disputes arising from the use of this Report shall be governed by and construed in accordance with the laws of the State of California, United States, without regard to its conflict of law provisions.

By using or relying on this Report in any way, you acknowledge that you have read, understood, and agree to be bound by this disclaimer.

⚠️ Issues

🔴 C-1: A liquidity provider can steal from pool via flash loan

(the flash loan functionality was also fully removed from the smart contract before publishing)

A liquidity provider can drain a pool by getting a flash loan.

When repaying a flash loan, the underlying value of the shares is artificially increased. It should be increased by `interest - interest_fee` but it is increased by `repay_amt - interest_fee` . Once the underlying value is artificially increased, any withdrawal of liquidity will have its value inflated.

```
un repay_flash_loan<T, ST>(
: &mut Pool<T, ST>, receipt: FlashLoanReceipt<T>, balance: Balance<T>)

repay_amt: u64 = calc_flash_loan_repay_amt(pool: pool, receipt: &receipt);
rt!(balance.value() == repay_amt, EInvalidRepayAmount);

interest: u64 = repay_amt - receipt.amount;
interest_fee: u64 = util::muldiv(a: interest, b: pool.interest_fee_bps as u64, c: 10000);

share_registry: &mut (kai_leverage=0x0)::equity::EquityRe… = pool.supply_equity.borrow_mut_regist
e_registry.increase_value(value: repay_amt - interest_fee);
ty::join(
self: &mut pool.collected_fees,
other: share_registry.increase_value_and_issue(value: interest_fee)

.available_balance.join(balance: balance);

FlashLoanReceipt { amount: _ } = receipt;
```

To replicate:

- ▶ Steps
- Github commit with test

pool flash loan test code

🔧 Recommendation

- The fix is the following rewrite: `repay_amt - interest_fee` ⇒ `interest - interest_fee`. This ensures that only the actual interest (minus fees) is added to the pool's value.
- We also suggest tracking flash loan liabilities separately from the pool's underlying value. This allows double-accounting and makes verification easier.

```
struct Pool<phantom T, phantom ST> has key { [...] flash_loan_liabilities: u64, } public fun flash_loan<T, ST> (pool: &mut Pool<T, ST>, amount: u64): (Balance<T>, FlashLoanReceipt<T>) { pool.flash_loan_liabilities = pool.flash_loan_liabilities + amount; [...] } public fun repay_flash_loan<T, ST>( pool: &mut Pool<T, ST>, receipt: FlashLoanReceipt<T>, balance: Balance<T>) { [...] pool.flash_loan_liabilities = pool.flash_loan_liabilities - receipt.amount; }
```

✅ Remediation

Commit dac0401

🔴 C-2: `max_liquidity_factor` computation is incorrect

(this issue was intentionally added to check audit quality)

This function calculates the maximum liquidation factor for a financial position based on its current margin level. The denominator computation for one case is incorrect: it uses `current_margin_x64` instead of `liq_margin_x64`.

```
fun calc_max_liq_factor_x64(
rent_margin_x64: u128, liq_margin_bps: u16, liq_bonus_bps: u16, base_liq_factor_b
 {
 current_margin_x64: u256 = current_margin_x64 as u256;
 liq_margin_x64: u256 = ((liq_margin_bps as u256) << 64) / 10000;
 crit_margin_x64: u256 = ((10000 << 64) + ((liq_bonus_bps as u256) << 64)) / 1000
 base_liq_factor_x64: u256 = ((base_liq_factor_bps as u256) << 64) / 10000;

for sanity, normally `liq_margin_x64 > crit_margin_x64`
 liq_margin_x64: u256 = util::max_u256(a: liq_margin_x64, b: crit_margin_x64);
 base_liq_factor_x64: u256 = util::min_u256(a: base_liq_factor_x64, b: 1 << 64);

(current_margin_x64 >= liq_margin_x64) {
 0
lse if (current_margin_x64 < crit_margin_x64) {
 // M / Mc
 ((current_margin_x64 << 64) / crit_margin_x64 as u128)
lse if (liq_margin_x64 - current_margin_x64 >= current_margin_x64 - crit_margin_x
 // M < (Ml + Mc) / 2
 1 << 64
lse {
 // (Ml - M + base_liq_factor * (M - Mc)) / (M - Mc)
 let num: u256 = (
     (liq_margin_x64 << 64) - (current_margin_x64 << 64) +
     base_liq_factor_x64 * (current_margin_x64 - crit_margin_x64)
 );
 let denom: u256 = (current_margin_x64 - crit_margin_x64);
 (num / denom as u128)
```

🔧 Recommendation

Correct the denominator computation:
`current_margin_x64` ⇒ `liq_margin_x64`.

✅ Remediation

Commit 1049813

🔴 C-3: Missing `assert` in `liquidate_col_x` / `liquidate_col_y`

(this issue was intentionally added to check audit quality)

The missing `assert` allows using an incorrect configuration, which may allow attacks such as liquidating a valid position and taking the reward. It is not clear whether an attacker could create all required objects for the attack, but it is worth fixing nevertheless.

```
package) macro fun liquidate_col_x<$X, $Y, $SY, $LP>(
sition: &mut Position<$X, $Y, $LP>,
nfig: &PositionConfig,
ice_info: &PythPriceInfo,
t_info: &DebtInfo,
payment: &mut Balance<$Y>,
pply_pool: &mut SupplyPool<$Y, $SY>,
ck: &Clock
ice<$X> {
 position = $position;
 config = $config;
 repayment = $repayment;
 supply_pool = $supply_pool;

k_versions(position, config);
rt!(position.ticket_active() == false); // ETicketActive
 price_info = validate_price_info(config, $price_info);
 debt_info = validate_debt_info(config, $debt_info);
```

```
assert!(position.config_id ==
  object::uid_to_inner(&config.id), EInvalidConfig);
```

🔧 Recommendation

Add `assert!(position.config_id() == object::id(config), EInvalidConfig);` to `liquidate_col_x` and `liquidate_col_y`.

✅ Remediation

Commit 1d5d1b8

🟡 L-1: Missing increment of `i`

The `update_interest` function `while` loop is missing an increment of `i` . We consider this a low-priority bug as it would most likely be caught in testing.

```
ıte the interest accrued since the last update and distribute the interest fee.
'un update_interest<T, ST>(pool: &mut Pool<T, ST>, clock: &Clock) {
dt: u64 = util::timestamp_sec(clock: clock) - pool.last_update_ts_sec;
dt == 0) {
return

utilization_bps: u64 = utilization_bps(pool: pool);

mut total_liabilities_x64: u128 = 0;
i: u64 = 0;
n: u64 = pool.debt_info.size();
.e (i < n) {
let (_, info: &mut (kai_leverage=0x0)::supply_pool::Len…) = pool.debt_info.get_e

let apr_bps: u64 = info.interest_model.value_at(x: utilization_bps);
let accrued_interest_x64: u128 = util::muldiv_u128(
    a: info.debt_registry.liability_value_x64(),
    b: (apr_bps as u128) * (dt as u128),
    c: 100_00 * SECONDS_IN_YEAR,
);
let fee_x64: u128 = util::muldiv_u128(a: accrued_interest_x64, b: pool.interest_

// increase supply shares underlying value by the accrued interest, and collect
let share_registry: &mut (kai_leverage=0x0)::equity::EquityRe… = pool.supply_equ
share_registry.increase_value_x64(value_x64: accrued_interest_x64 - fee_x64);
equity::join(
    self: &mut pool.collected_fees,
    other: equity::increase_value_and_issue_x64(registry: share_registry, value_
);

// increase debt shares liability by the accrued interest
info.debt_registry.increase_liability_x64(value_x64: accrued_interest_x64);

total_liabilities_x64 = total_liabilities_x64 + info.debt_registry.liability_val
```

missing increment of i

🔧 Recommendation

Increment `i` in the `while` loop.

✅ Remediation

Commit 5da0232

🟡 L-2: Bug in condition when checking we borrowed enough

It checks for `borrowed_x` while the function refers to position `y` (github) . It is not critical because `create_position` rechecks correctly.

```
package) macro fun borrow_for_position_y<$X, $Y, $SY, $I32>(
ket: &mut CreatePositionTicket<$X, $Y, $I32>,
fig: &PositionConfig,
pply_pool: &mut SupplyPool<$Y, $SY>,
ck: &Clock

ticket = $ticket;
config = $config;
supply_pool = $supply_pool;

rt!(ticket.config_id() == object::id(config)); // EInvalidConfig
ticket.dy() == ticket.borrowed_x().value()) {
return
```

🔧 Recommendation

Rewrite `ticket.borrowed_x()` ⇒ `ticket.borrowed_y()` in the early return condition of `borrow_for_position_y`.

✅ Remediation

Commit 4993c58

🟡 L-3: Incorrect treasury empty check

The treasury check is incomplete in `add_lend_facil` (github). The condition should be `registry.supply_x64() == 0 && registry.underlying_value_x64() == 0`.

`create_pool` is an example of the correct condition (github).

Not major because it is a function used by the owner of the pool — it just protects against silly mistakes.

```
un add_lend_facil<T, ST>(
.: &mut Pool<T, ST>, facil_id: ID, interest_model: Piecewise, debt_registry: DebtRegis
 &mut TxContext
nRequest {
rt!(debt::supply_x64(registry: &debt_registry) == 0, EDebtTreasuryNotEmpty);
..debt_info.insert(
key: facil_id,
value: LendFacilInfo { interest_model, debt_registry, allow_borrow: true }


ss::new_request(_: AConfigLendFacil { }, ctx: ctx)
```

🔧 Recommendation

Add `registry.supply_x64() == 0` to the assert `EDebtTreasuryNotEmpty` in `add_lend_facil`.

✅ Remediation

Commit 1ace884

🧹 R-1: Split `debt_bag` into its two components

`debt_bag`, of type `FacilDebtBag`, always holds exactly two values. We suggest we split it into `shares_x` and `shares_y`, of type `FacilDebtShare`. This both simplifies the code and improve type safety.

🔧 Recommendation

```
osition<phantom X, phantom Y> has key {
osition<phantom X, phantom Y, phantom SX, phantom SY> has key {
JID,
ig_id: ID,
osition: CetusPosition,
x: Balance<X>,
y: Balance<Y>,
_bag: FacilDebtBag,
es_x: FacilDebtShare<SX>,
es_y: FacilDebtShare<SY>,
ected_fees: BalanceBag,
et_active: bool,
ion: u16,
```

☐ Remediation

Suggestion not integrated.

⊢ Verification summary

We formally verified several properties of `supply_pool.move` and `position_model.move`. The verification led to identifying several of the discovered bugs and vulnerabilities. This is a summary of the specs.

`supply_pool.move`

Rounding

The initial goal was to ensure that the rounding is done correctly for `EquityTreasury` and `DebtTreasury`. The added specs led to discovering other vulnerabilities as well.

- for `EquityTreasury` we want the price of a share to always increase
  - we cannot prove this directly in the `equity` module because there are functions (`decrease_value` and `decrease_value_x64`) which allows taking value out, which decreases the value of all the shares
  - instead **we proved that the price always goes up for all uses of `EquityTreasury` in `supply_pool`**
  - we did this by adding specs for all functions using `EquityTreasury`
- for `DebtTreasury` we want the rounding to go into the opposite direction
  - whenever someone deposits or withdraws, the price either stays the same or goes down
  - when adding the interest, the price of a share goes up
  - **we proved that it is never the case that there are 0 debt shares but there is still outstanding debt** (`supply_x64 == 0 ==> liability_value_x64 == 0`)

Bookkeeping property

For the `SupplyPool`, we want the balance (underlying value) to be equal to assets minus liabilities. Adding this spec led us to the flash loan critical vulnerability (C-1).

`position_model.move`

Avoiding sensitive aborts